

AWS – REFACTOR

1. Introduction

In this project, we refactor the **vProfile application** for cloud-native deployment on AWS, moving beyond the traditional lift-and-shift approach. The goal is to create a **flexible, scalable, and low-maintenance infrastructure** using **Platform-as-a-Service (PaaS)** and **Software-as-a-Service (SaaS)** offerings instead of manually managing EC2 instances.

Objectives

- Boost **agility** and **business continuity**.
- Enable **easy scaling** and improved **performance**.
- Reduce **operational overhead** and dependency on large teams.
- Implement **Infrastructure as Code (IaC)** and automation wherever possible.
- Adopt a **pay-as-you-go** cloud model.

Key AWS Services Used

Component	AWS Service	Purpose
Application Hosting	Elastic Beanstalk	Hosts Tomcat applications with auto-scaling, load balancing, and EC2 management.
Database	Amazon RDS	Managed relational database with automatic backups and scaling.
Caching	ElastiCache	Replaces Memcache for high-performance caching.
Messaging	Amazon MQ	Replaces RabbitMQ for message brokering.
Storage	S3 / EFS	Artifact storage and shared file system.
DNS	Route 53	Domain name resolution and routing.
Content Delivery	CloudFront	Global content delivery with caching and SSL support.
Monitoring	CloudWatch	Monitoring, logging, and alarms for auto-scaling and performance.

Architecture Overview

1. User accesses the application URL via **Route 53**.
2. Requests are served through **CloudFront CDN** for global performance.
3. Traffic reaches **Elastic Beanstalk's Application Load Balancer**, which forwards it to EC2 instances in an **auto-scaling group** running Tomcat.
4. Frontend communicates with backend services: **RDS**, **ElastiCache**, and **Amazon MQ**.
5. **CloudWatch** monitors the environment and triggers auto-scaling based on demand.
6. Artifacts are stored in **S3**, enabling easy deployment updates.

Deployment Workflow

1. Create **key pair** and **security groups** for backend services.
2. Launch **RDS**, **ElastiCache**, and **Amazon MQ**.
3. Initialize the database from a temporary EC2 instance.
4. Set up **Elastic Beanstalk** environment with HTTPS listener and `/login` health check.
5. Build and deploy application artifact with backend service endpoints.
6. Configure **CloudFront** for content delivery and SSL.
7. Update DNS records via **Route 53** or external provider.
8. Test the application from the final URL.

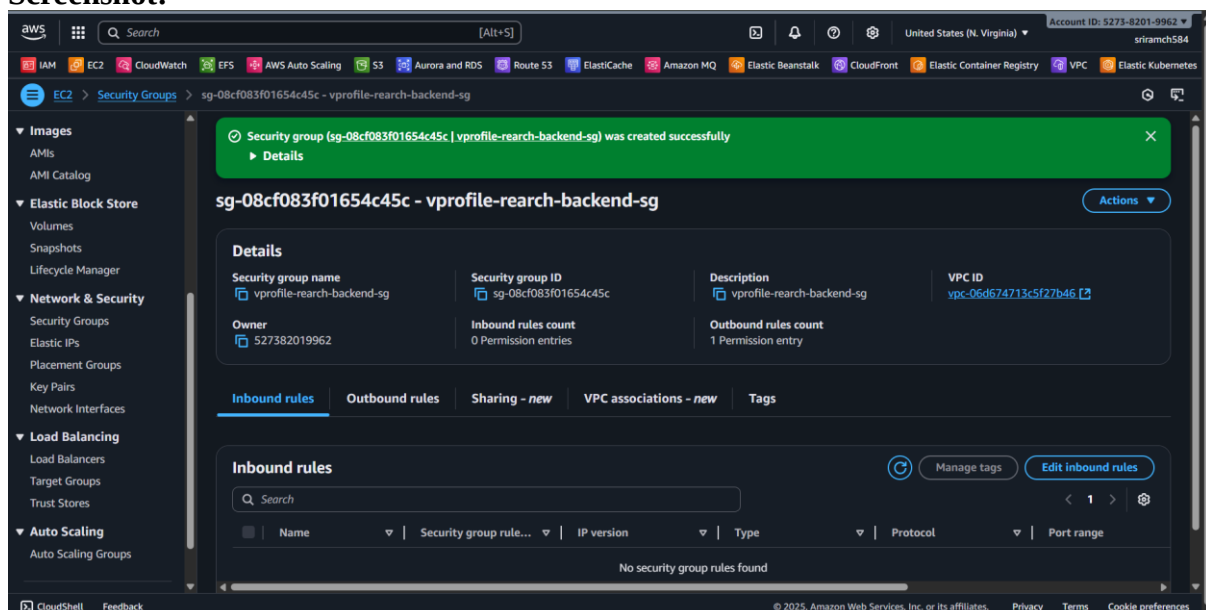
2. Security Groups and Key Pairs

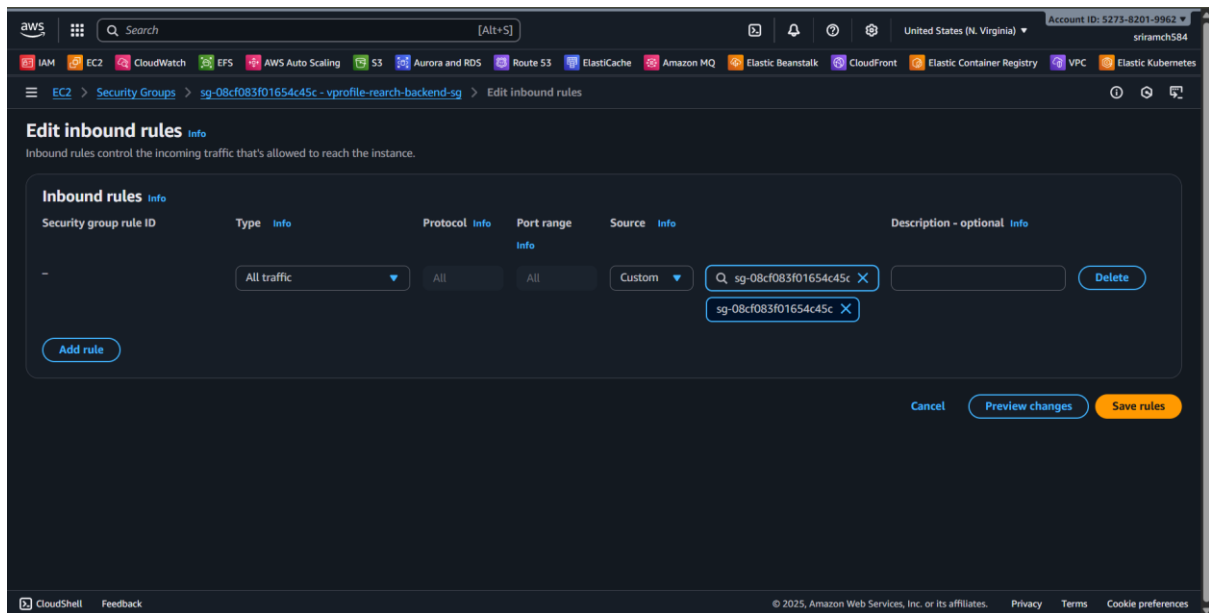
In our re-architected **vProfile application stack**, proper access control and administrative access are essential. This section outlines the setup of **security groups** and **key pairs**.

Security Groups

- **Elastic Beanstalk EC2 Instances:**
 - Elastic Beanstalk automatically creates a **security group** for the EC2 instances hosting our application.
 - It also creates a **load balancer security group** and configures rules to allow traffic from the load balancer to the application instances.
- **Backend Services (RDS, ElastiCache, Amazon MQ):**
 - A dedicated **backend security group** is created for all backend services.
 - An **inbound rule** is added to allow **all traffic within the group**, enabling communication between backend services.
 - After Beanstalk deployment, the backend security group is updated to allow traffic from the Beanstalk security group.

Screenshot:

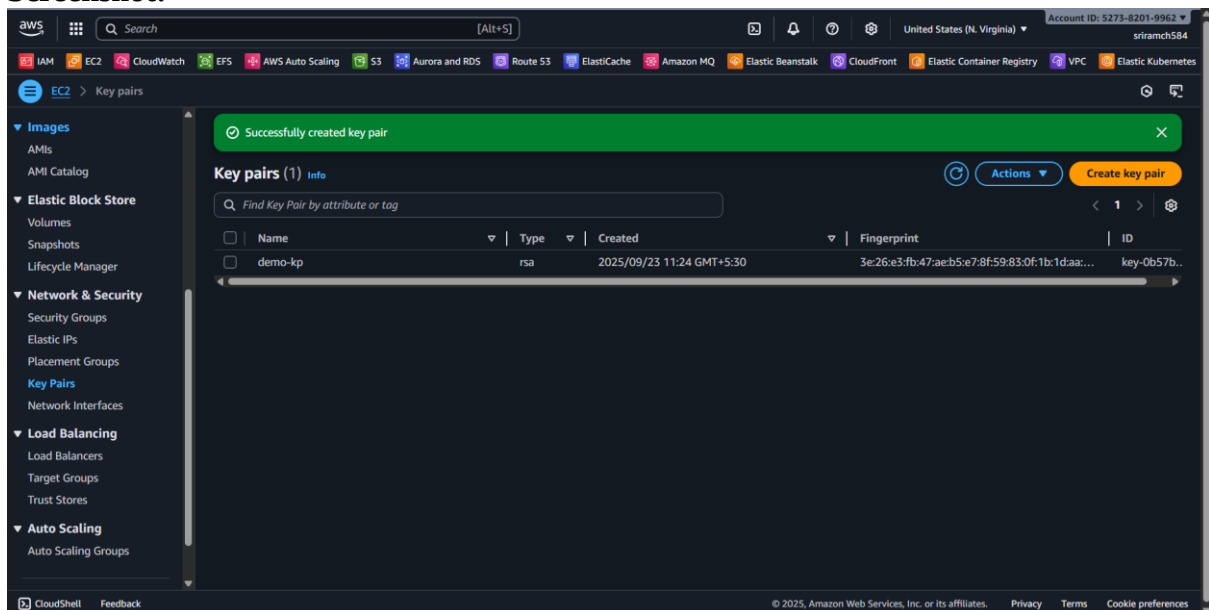




Key Pairs

- A **key pair** is created for Beanstalk EC2 instances to enable SSH login for troubleshooting.
- While Beanstalk handles deployments automatically, the key pair provides **administrative access** if needed.

Screenshot:



3. RDS (Relational Database Service)

For the **vProfile application**, we use **Amazon RDS** to host our MySQL database, providing a fully managed, secure, and scalable database service.

Key Steps in Setting Up RDS:

1. **Parameter Groups**
 - RDS uses **parameter groups** to manage database configurations.
 - A **custom parameter group** (vProfile-RDS-Rearch-ParaGroup) is created for MySQL 8 to allow future configuration changes without SSH access to the database.
2. **Subnet Groups**
 - A **DB subnet group** defines which subnets in the VPC the database can use.
 - A dedicated subnet group (vProfile-RDS-Rearch-SubnetGroup) ensures better control and security over the database network.
3. **Creating the RDS Instance**
 - **Engine:** MySQL 8.0 (Community Edition)
 - **Template:** Free tier (single instance)
 - **DB Instance Identifier:** vProfile-RDS-Arch
 - **Master Username:** admin (auto-generated password)
 - **Instance Type:** db.t4g.micro (free tier)
 - **Storage:** 20 GB GP3, auto-scaling disabled
 - **VPC & Subnet Group:** Use the custom subnet group
 - **Public Access:** Disabled (internal communication only)
 - **Security Group:** vProfile-BackEnd-SG
 - **Port:** 3306 (default MySQL port)
4. **Creating the Application Database**
 - A dedicated database (vProfileDB) is created in the RDS instance to store the profile application data.
 - This database name must match the application configuration in the **properties file**.
5. **Credentials and Endpoint**
 - Save the **username, password, and RDS endpoint** for use in the application properties.
 - If the password is lost, it can be **reset**, but the existing password cannot be recovered.
6. **Optional Configuration**
 - Automated backups, log export to CloudWatch, and maintenance settings can be enabled depending on project requirements.

4. ElastiCache (Memcached)

For the **vProfile application**, we use **Amazon ElastiCache** to provide a fully managed **in-memory caching service** using **Memcached**, improving application performance by reducing database load.

Key Steps in Setting Up ElastiCache

1. **Parameter Groups**
 - ElastiCache uses **parameter groups** to manage cache configuration.
 - A custom parameter group (vProfile-Rearch-Cache-Parameter-Group) is created for **Memcached 1.6**, allowing future configuration changes.
2. **Subnet Groups**

- A **cache subnet group** defines which subnets in the VPC the cache nodes can use.
 - A dedicated subnet group ensures proper network control and security for caching services.
3. **Creating the ElastiCache Cluster**
 - **Engine:** Memcached 1.6
 - **Node Type:** `cache.t2.micro` (smallest free tier-compatible node, 0.5 GB RAM)
 - **Number of Nodes:** 1
 - **Port:** 11211 (default for Memcached, must match application properties)
 - **Parameter Group:** `vProfile-Rearch-Cache-Parameter-Group`
 - **Subnet Group:** Previously created subnet group
 - **Security Group:** `vProfile-BackEnd-SG` (same as backend services)
 - **Availability Zone:** No preference (single node)
 4. **Access and Endpoint**
 - After creation, an **endpoint** is provided.
 - This endpoint is used in the **application properties file** for caching integration.

5. Amazon MQ (RabbitMQ)

For the **vProfile application**, we use **Amazon MQ** to provide a fully managed **message broker service**, replacing self-managed RabbitMQ on EC2. This allows reliable communication between application components with minimal operational overhead.

Key Steps in Setting Up Amazon MQ

1. **Service Selection**
 - Navigate to **Amazon MQ** in AWS Console.
 - Select **RabbitMQ** as the broker engine.
2. **Broker Configuration**
 - **Deployment Mode:** Single instance (saves cost; clustering optional for production).
 - **Broker Name:** `vProfile-Rearch-RabbitMQ`
 - **Instance Type:** `mq.t3.micro` (smallest free-tier compatible instance).
 - **Username/Password:** Set credentials for application integration (save securely for application properties file).
3. **Additional Settings**
 - **Broker Engine Version:** Use default (e.g., 3.13).
 - **Access Type:** Private (internal communication within the VPC).
 - **VPC/Subnet:** Default VPC and subnet are used.
 - **Security Group:** `vProfile-BackEnd-SG` (allows secure communication with backend services).
 - **Encryption & Maintenance:** Default settings; CloudWatch logs optional.
4. **Access and Endpoint**
 - After creation, Amazon MQ provides an **endpoint** for integration with the application.

- The endpoint is referenced in the **application properties file** for message queue communication.

6. Elastic Beanstalk

For the **vProfile application**, **AWS Elastic Beanstalk** is used to deploy and manage the application effortlessly. It provides a **fully managed environment**, handling infrastructure, auto-scaling, load balancing, and deployment automation.

Key Features

- **Deployment Environment:** Web server environment using **Tomcat** with **Java 21 (Corretto 21)**.
- **Automation:** Automatically sets up **EC2 instances, auto-scaling group, load balancer, S3 bucket for artifacts, monitoring, and logs**.
- **Cost Efficiency:** No additional charge for Beanstalk; you pay only for underlying AWS resources like EC2 and storage.
- **Role Management:** Custom **IAM roles** created for service access to ensure proper permissions.

Environment Configuration

1. **Application & Environment Names:** Unique identifiers for access via URL.
2. **Platform:** Tomcat 10 with Java 21 (Corretto 21).
3. **Capacity & Scaling:**
 - Minimum 2 instances, maximum 4.
 - Auto-scaling based on **CPU utilization** or **network traffic**.
4. **Instance Type:** T2 micro (free-tier compatible).
5. **Load Balancer:** Application load balancer with **stickiness enabled** for session persistence.
6. **Storage:** Root volume set to **GP3** for compatibility with launch templates.
7. **Health & Monitoring:** Enhanced monitoring enabled to manage deployments safely.
8. **Deployment Policies:** Supports **rolling, rolling with additional batch, immutable, and traffic splitting**. For the project, **rolling deployment with 50% batch size** is used.
9. **Environment Properties:** Optional configuration variables for database or other services.

7. Update on Security Group & ELB

With the **Elastic Beanstalk environment** up and running, the default application is accessible via the provided Beanstalk URL. In the next steps, this will be replaced by the **vProfile application**.

Backend Connectivity

The Beanstalk environment must securely connect to backend services:

- **RDS Database**
- **Amazon MQ (RabbitMQ)**
- **ElastiCache (Memcache)**

These services are part of the **vProfile backend security group**. To ensure connectivity:

1. Identify the **instance security group** of Beanstalk (not the load balancer's security group).
2. Add an **inbound rule** to the backend security group:
 - Allow all traffic **from the Beanstalk instance security group**.
 - Add a description for clarity, e.g., *"Allow traffic from Beanstalk instance SG"*.

Preparation for Deployment

- Collect the **endpoints, ports, usernames, and passwords** of all backend services.
- Store them in a file for configuration during the **artifact deployment** to Beanstalk.

8. Build & Deploy Artifact

After setting up the backend services (RDS, Amazon MQ, and ElastiCache) and the Elastic Beanstalk environment, the next step is to **build the application artifact and deploy it**.

Steps Overview

1. **Collect Backend Service Details**
 - RDS endpoint, username, and password
 - RabbitMQ endpoint, port, username, and password.
 - Memcache endpoint and port.
2. **Update Application Configuration**
 - Edit `Application.properties` to include the backend service details.
 - Ensure all endpoints, ports, usernames, and passwords are correct.
3. **Clone Source Code and Build Artifact**
 - Clone the repository from GitHub.
 - Switch to the `AWS refactor` branch.
 - Build the project using Maven (`mvn install`).
 - Verify the artifact is generated in the `target` folder.
4. **Deploy to Elastic Beanstalk**
 - Upload the generated artifact (`.war` file) to Beanstalk.
 - Assign a version name and deploy using **rolling deployment** (batch size 50%).
 - Monitor deployment progress in the Beanstalk events section.
5. **Verify Application**
 - Access the application URL via beanstalk endpoint url
 - Log in with configured credentials to verify database and backend service connectivity.

- Confirm that caching and messaging services (Memcache, RabbitMQ) are working correctly.